



# OASIS Security Report

## Local File Inclusion Security Analysis

**Date:** 2026-06-12 00:40:06 — **Model:** llama3.2:3b

**Vulnerability:** Local File Inclusion

**Project:** vulnerable

**Analysis root:** `../vulnerable`

### Table of contents

□ Summary

□ Statistics

□ Files

□ Detailed analysis

□ Assistant

△ Errors & notes

## Summary

Analyzed **3** file(s) for Local File Inclusion.

[Return to table of contents](#)

## Statistics

**Files analyzed:** 3    **Total findings:** 3

|          |             |   |
|----------|-------------|---|
| Critical | <div></div> | 3 |
| High     | <div></div> | 0 |
|          | <div></div> |   |

**Medium****0****Low****0**[Return to table of contents](#)

## Files

| File                                                                                           | Similarity |
|------------------------------------------------------------------------------------------------|------------|
| /work/project/dataset/vulnerable/reentrancy/etherstore.sol                                     | 0.624      |
| /work/project/dataset/vulnerable/reentrancy/<br>0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol | 0.619      |
| /work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol                        | 0.591      |

[Return to table of contents](#)

## Detailed analysis

### File 1: /work/project/dataset/vulnerable/reentrancy/etherstore.sol

Similarity score: 0.624

Source lines: 1-39

#### ► Finding 1.1: Local File Inclusion (Critical)

##### Vulnerable code

```
require(msg.sender.call.value(_weiToWithdraw()));
```

##### Explanation

The require statement allows for file inclusion via the msg.sender call.

##### Impact

Source code disclosure and potential remote code execution

##### Entry point

/withdrawFunds

##### HTTP methods

POST

##### Parameters

\_weiToWithdraw

##### Exploitation steps

- Submit a large value for \_weiToWithdraw to cause the require statement to expand the file inclusion.
- Use a malicious path in the msg.sender call to include and execute arbitrary files.

##### Example payloads

- `_weiToWithdraw = 'open(basedir)/etc/passwd'`
- `_weiToWithdraw = '../etc/passwd'`

##### Conditions

msg.sender is authenticated.

## Remediation

Use the `file\_inclusion` guard to validate the path before including it.

### Secure example

```
require(file_inclusion);
```

## File 2: /work/project/dataset/vulnerable/reentrancy/0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol

Similarity score: 0.619

Source lines: 1-85

### ► Finding 2.1: Local File Inclusion (Critical)

#### Vulnerable code

```
$( _am ) ( )
```

#### Explanation

The `call` function is vulnerable to Local File Inclusion because it executes user-provided input without proper validation or sanitization.

#### Impact

Potential source code disclosure, configuration exposure, and credential theft.

#### Entry point

/CashOut(uint \_am)

#### Execution path

```
GET /CashOut \u003f\_am()\nPOST /CashOut \u003f\_am()\n
```

#### HTTP methods

GET, POST

#### Parameters

\_am

#### Exploitation steps

- Make a request to /CashOut \u003f\\_am() with user-controlled input

#### Example payloads

- `$( _am ) = 'C:\Windows\system32\cmd.exe'`

#### HTTP raw requests (Burp / ZAP)

```
GET /CashOut?_am=C%3A%5Cwindows%5Csystem32cmd.exe HTTP/1.1\r\nHost:
example.com\r\n\r\n\r\n\u003f_%7Am
```

## File 3: /work/project/dataset/vulnerable/access-control/parity\_wallet\_bug\_2.sol

Similarity score: 0.591

Source lines: 96-414

### ► Finding 3.1: Local File Inclusion (Critical)

#### Vulnerable code

```
function confirm(bytes32 _h) onlymanyowners(_h) returns (bool o_success) {
    if (m_txs[_h].to != 0 || m_txs[_h].value != 0 || m_txs[_h].data.length != 0) {
        address created;
        if (m_txs[_h].to == 0) {
            created = create(m_txs[_h].value, m_txs[_h].data);
        }
        else
        {
            created = _to.call.value(m_txs[_h].value)(_txs[_h].data);
        }
        MultiTransact(msg.sender, _h, m_txs[_h].value, m_txs[_h].to, m_txs[_h].data,
            created);
        delete m_txs[_h];
        return true;
    }
}
```

#### Explanation

The `confirm` function uses user-controlled input `\_h` to access `m\_txs`, which allows for potential Local File Inclusion attacks.

#### Impact

Source code disclosure and configuration exposure

#### Entry point

/confirm(bytes32 \_h)

#### Execution path

```
confirm(bytes32 _h) {
    if (m_txs[_h].to != 0 || m_txs[_h].value != 0 || m_txs[_h].data.length != 0)
    {...
    }
```

#### Parameters

\_h

## Exploitation steps

- Pass a specially crafted `\_h` value to `confirm` to access sensitive files.

## Example payloads

- `sha3('path/to/sensitive/file')`

## HTTP raw requests (Burp / ZAP)

```
GET /confirm?_h=sha3%27%2Fpath%2Fto%2Fsensitive%2Ffile HTTP/1.1
Host: example.com
Accept: */*
```

[Return to table of contents](#)

## Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

## Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)